image_fusion.py

```python
#%%
import cv2
import numpy as np

#%%
img_a = cv2.imread("a.jpg")
img_b = cv2.imread("b.jpg")
# 将图像转换为灰度图
gray_a = cv2.cvtColor(img_a, cv2.COLOR_BGR2GRAY)
gray_b = cv2.cvtColor(img_b, cv2.COLOR_BGR2GRAY)

# 初始化SIFT检测器并提取特征
sift = cv2.SIFT_create()
kp_a, des_a = sift.detectAndCompute(gray_a, None)  # 图像a的关键点和描述符
kp_b, des_b = sift.detectAndCompute(gray_b, None)  # 图像b的关键点和描述符

# 使用FLANN匹配器进行特征匹配
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)  # 检查次数越多，匹配越准确但速度越慢
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des_a, des_b, k=2)  # k=2表示每个特征点返回2个最佳匹配

# 应用Lowe's比率测试筛选优质匹配点
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:  # 比率阈值通常取0.7-0.8
        good_matches.append(m)

# 绘制匹配的SIFT关键点
matched_keypoints_img = cv2.drawMatches(
    img_a, kp_a, img_b, kp_b, good_matches,
    None, flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# 提取匹配点的坐标
src_pts = np.float32([kp_b[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)  # 图像b的关键点
dst_pts = np.float32([kp_a[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)  # 图像a的关键点

# 使用RANSAC算法估计单应矩阵（透视变换矩阵）
H, _ = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

# 获取输入图像尺寸
h_a, w_a = img_a.shape[:2]
h_b, w_b = img_b.shape[:2]

# 计算图像b变换后的四个角点坐标
pts = np.float32([[0, 0], [0, h_b], [w_b, h_b], [w_b, 0]]).reshape(-1, 1, 2)
dst_corners = cv2.perspectiveTransform(pts, H)

# 确定拼接后图像的最终尺寸（包含所有像素）
```

```
all_corners = np.concatenate([dst_corners, np.float32([[0,0], [w_a,0], [w_a,h_a], [0,h_a]]).reshape(-1,1,2)], axis=0)
[x_min, y_min] = np.int32(all_corners.min(axis=0).ravel() - 0.5)
[x_max, y_max] = np.int32(all_corners.max(axis=0).ravel() + 0.5)

# 创建平移矩阵，确保所有像素都在可见区域内
translation_matrix = np.array([[1, 0, -x_min], [0, 1, -y_min], [0, 0, 1]], dtype=np.float32)

# 对图像b进行透视变换和平移
stitched_img = cv2.warpPerspective(
    img_b,
    translation_matrix @ H,  # 组合平移矩阵和单应矩阵
    (x_max - x_min, y_max - y_min)  # 输出图像尺寸
)

# 将图像a复制到拼接结果的对应位置
stitched_img[-y_min : h_a - y_min, -x_min : w_a - x_min] = img_a


# 显示匹配关键点和拼接结果
cv2.imshow('Matched SIFT Keypoints', matched_keypoints_img)  # 显示匹配的关键点
cv2.imshow('Stitched Result', stitched_img)                  # 显示拼接后的图像
cv2.waitKey(0)  # 等待用户按键（按任意键关闭窗口）
cv2.destroyAllWindows()  # 关闭所有显示窗口

# 保存拼接结果
# cv2.imwrite("stitched_result.jpg", stitched_img)
# print("拼接完成，结果已保存为 stitched_result.jpg")

# %%
```